Architecture & Engineering Report: Dr. Lowkey Assistant

1. Executive Summary

The Dr. Lowkey Medical Assistant is a robust Retrieval-Augmented Generation (RAG) system engineered to provide safe, evidence-based health information. Built on a FastAPI backend, the architecture leverages a hybrid retrieval engine using a local Qdrant database, dual-embedding models via FastEmbed, and the Gemini 3.5 Flash Lite model for rapid answer generation. To ensure strict user safety, the system implements multi-tier logic guardrails that intercept emergency, harmful, and conversational queries before they reach the retrieval engine.

2. Data Pipeline & Ingestion

- Web Scraping: The system uses 'requests' and 'BeautifulSoup' to scrape HTML pages from verified sources (NHS, CDC). It strips navigation noise and converts the core HTML to Markdown.
- PDF Extraction: Medical PDFs are downloaded and parsed using 'pypdf', extracting page-by-page text into Markdown format.
- Chunking Strategy: Ingested text is divided into chunks of exactly 500 characters, with a 50-character overlap to preserve semantic context across paragraph boundaries.
- Database Upsertion: Payloads are indexed in batches of 200 into a Qdrant collection named 'medical_knowledge'.

3. Hybrid Retrieval & Reranking Engine

- Dual Embeddings: The system uses FastEmbed to generate two vector types for every chunk: dense embeddings (BAAI/bge-small-en-v1.5) and sparse embeddings (Qdrant/bm25).
- Hybrid Search: Queries are embedded using both models and searched against Qdrant using Reciprocal Rank Fusion (RRF), fetching the top 10 most relevant chunks.
- Cross-Encoder Reranking: The initial 10 results pass through FlashRank (ms-marco-MiniLM-L-12-v2) to surface the absolute top 3 chunks, ensuring maximum precision for the LLM context window.

4. Design Decisions & Trade-offs

- Hybrid Search vs. Dense Only: While dense embeddings capture semantic meaning, medical queries often rely on exact terminology and acronyms. Adding BM25 sparse vectors ensures high recall for specific medical jargon that dense models might miss.
- Architectural Efficiency: The pipeline utilizes quantized local embedding models and cross-encoders rather than heavy LLM-based rerankers. This minimizes latency and compute overhead, ensuring the software

architecture remains lightweight enough to eventually integrate with embedded electronics or local medical instrumentation, bridging the gap between AI software and hardware constraints.

- Generative Model Choice: Gemini 3.5 Flash Lite was selected over heavier models to prioritize rapid token streaming for a seamless UI experience while remaining highly cost-effective.

5. Multi-Tiered Safety & Routing Logic

- Priority Safety Bypass: A regex-based intent matcher scans for red-flag topics (self-harm, violence, dangerous drug usage). If detected, it bypasses the RAG pipeline entirely and returns a hardcoded, caring safety response.
- Conversational Bypass: Casual inputs (greetings, gratitude) are intercepted via regex, preventing the system from executing unnecessary database searches and returning 'insufficient evidence' errors.
- Clinical Triage Guard: Evaluates standard medical queries against predefined emergency keywords. If triggered, the system alerts the user of a critical medical emergency and advises calling local emergency services.

6. Challenges Faced & Solutions

- The Conversational UX Trap: Initially, treating every single query as a vector database search broke the user experience when chatting naturally. I solved this by engineering the Regex intent matcher to route queries intelligently.
- UI Cache Invalidation: During rapid iterative frontend development, the browser stubbornly cached old CSS files. I implemented a cache-busting versioning strategy (?v=105) in the HTML header to force immediate UI updates.
- LLM Hallucination Control: To prevent the model from answering medical queries using its pre-trained knowledge, I utilized strict prompt engineering. The system instructions command the model to answer strictly using provided evidence chunks and to cite sources using inline numerical formats.

7. Future Scope

- Contextual Memory: Implementing session-based memory using Redis to allow the assistant to remember previous symptoms mentioned in a continuous diagnostic conversation.
- Quantitative Evaluation: Integrating frameworks like RAGAS to programmatically score the retrieval accuracy (Context Precision and Context Recall) of the hybrid pipeline.
- Multi-Modal Capabilities: Expanding the ingestion pipeline to process medical charts and images alongside text guidelines.